

Project Linked List

COS2103

Data structure and Algorithm

1 / 2568

Present to

Asst.Prof.DR.Chouvalit Khancome

By Krittaya Tantichaiyakul

6705500269

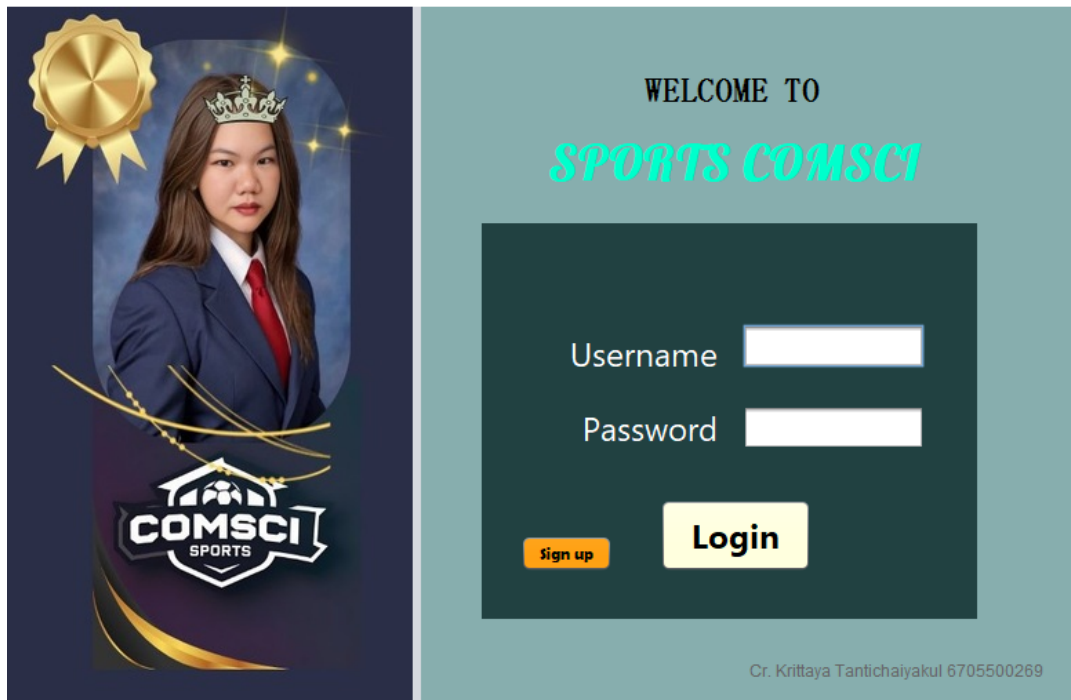
หัวข้อ

อุปกรณ์กีฬา



Program Interface Preview

1. Login page - Login



The login page features a dark blue background on the left with a portrait of a woman wearing a crown and a COMSCI SPORTS logo. The right side has a light blue background with the text 'WELCOME TO' and 'SPORTS COMSCI' in green. Below this is a dark green box containing input fields for 'Username' and 'Password', a 'Login' button, and a 'Sign up' button. At the bottom right, the text 'Cr. Krittaya Tantichaiyakul 6705500269' is displayed.

WELCOME TO

SPORTS COMSCI

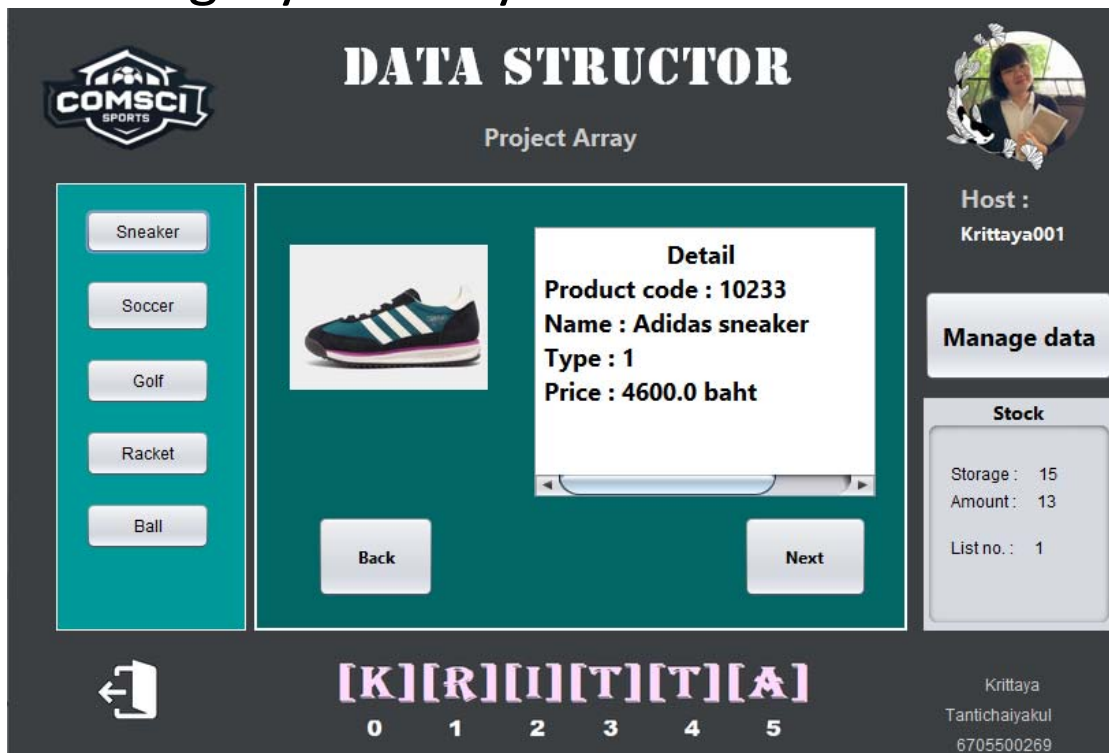
Username

Password

[Sign up](#) [Login](#)

Cr. Krittaya Tantichaiyakul 6705500269

2. Category of array - Menu



The menu interface has a dark grey background. At the top left is the COMSCI SPORTS logo. The title 'DATA STRUCTOR' is in large white letters, with 'Project Array' below it. On the right is a circular profile picture of a woman, the text 'Host : Krittaya001', a 'Manage data' button, and a 'Stock' section with 'Storage : 15', 'Amount : 13', and 'List no. : 1'. The main area has a teal background with a list of categories on the left: Sneaker, Soccer, Golf, Racket, and Ball. The 'Sneaker' category is selected, showing an image of a sneaker and a 'Detail' box with 'Product code : 10233', 'Name : Adidas sneaker', 'Type : 1', and 'Price : 4600.0 baht'. Below the detail box are 'Back' and 'Next' buttons. At the bottom, there is a navigation bar with a back arrow, a sequence of buttons labeled [K][R][I][T][T][A], and a row of numbers 0 through 5. The name 'Krittaya Tantichaiyakul 6705500269' is at the bottom right.

COMSCI SPORTS

DATA STRUCTOR

Project Array

Host : Krittaya001

[Manage data](#)

Stock

Storage : 15
Amount : 13
List no. : 1

Sneaker
Soccer
Golf
Racket
Ball

Detail
Product code : 10233
Name : Adidas sneaker
Type : 1
Price : 4600.0 baht

[Back](#) [Next](#)

[←](#) [K][R][I][T][T][A]

0 1 2 3 4 5

Krittaya
Tantichaiyakul
6705500269

3. list of products – Manage

DATA STRUCTOR AND ALGORITHM

Delete
Insert

Delete code

Status :

Code :

Name :

Price :

Type :

Delete

search code :

SHOW ALL

Host :
Krittaya001

Stock

Storage : -

Amount : 13

No.	Code	Name	Price	Type	Address
1	10233	Adidas sne...	4600.0	1	/sc/sneake...
2	10472	Nike sneaker	4200.0	1	/sc/sneake...
3	10785	Skecher sn...	2500.0	1	/sc/ske.jpg
4	20096	Umbro Glove	800.0	2	/sc/ub.jpg
5	20504	Nike footba...	1500.0	2	/sc/shirtNik...
6	30319	Ping stick g...	1300.0	3	/sc/golfstic...
7	30873	Anyday golf ...	5000.0	3	/sc/golfbag....
8	40152	Yonex strike	2400.0	4	/sc/yon.png
9	40197	Kawasaki s...	1700.0	4	/sc/kawa.jpg
10	40601	Victor shuttl...	200.0	4	/sc/shuttle.j...
11	50290	Puma footb...	800.0	5	/sc/puma.jpg
12	50475	Mikasa voll...	1200.0	5	/sc/volley.jpg

Present to Asst.Prof.DR.Chouvalit Khancome

by.. Krittaya Tantichaiyakul 6705500269

4. ไฟล์กลางจัดเก็บลิงค์ลิส – DataStore

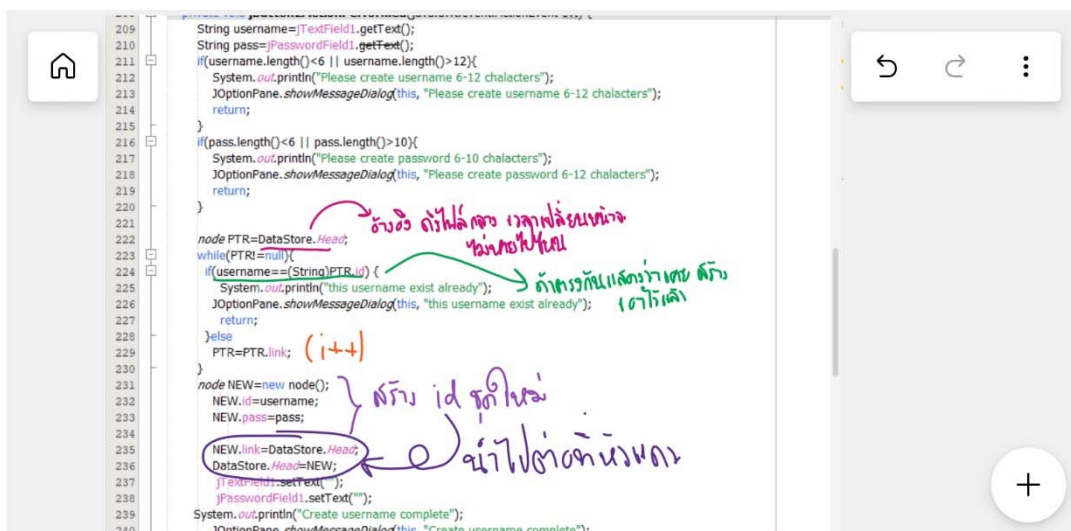
```

10  //
11  public class DataStore {
12      public static int n=13;
13      static DNode START, LAST;
14      public static DNode Head=null;
15      public static class node{
16          public String id;
17          public String pass;
18          public node link;
19      }
20      public static class DNode {
21          public int code;
22          public String name;
23          public int type;
24          public float price;
25          public String pic;
26          DNode BACK;
27          DNode FORW;
28      }
29      public void setAll(int a, String b, int c, float d, String e){
30          code = a;
31          name = b;
32          type = c;
33          price = d;
34          pic = e;
35      }
36      static int getIK(){
37          return n;
38      }
39  }
40
41  // ฟังก์ชันเพิ่มข้อมูลสินค้า
42  DNode NEW1= new DNode();
43  NEW1.setAll(10233,"Adidas sneaker",1,4600,"/sc/sneakerAdi.jpg");
44  START=NEW1;
45
46  DNode NEW2= new DNode();
47  NEW2.setAll(10472,"Nike sneaker",1,4200,"/sc/sneakerNike.jpg");
48  NEW3.setAll(10785,"Skecher sneaker",1,2500,"/sc/ske.jpg");
49  DNode NEW4= new DNode();
50  NEW4.setAll(20096,"Umbro Glove",2,800,"/sc/ub.jpg");
51  DNode NEW5= new DNode();
52  NEW5.setAll(20504,"Nike football shirt",2,1500,"/sc/shirtNike.jpg");
53  DNode NEW6= new DNode();
54  NEW6.setAll(30319,"Ping stick golf",3,1300,"/sc/golfstick.jpg");
55  DNode NEW7= new DNode();
56  NEW7.setAll(30873,"Anyday golf bag",3,5000,"/sc/golfbag.jpg");
57  DNode NEW8= new DNode();
58  NEW8.setAll(40152,"Yonex strike",4,2400,"/sc/yon.png");
59  DNode NEW9= new DNode();
60  NEW9.setAll(40197,"Kawasaki strike",4,1700,"/sc/kawa.jpg");
61  DNode NEW10= new DNode();
62  NEW10.setAll(40601,"Victor shuttlecock",4,200,"/sc/shuttle.jpg");
63  DNode NEW11= new DNode();
64  NEW11.setAll(50290,"Puma football",5,800,"/sc/puma.jpg");
65  DNode NEW12= new DNode();
66  NEW12.setAll(50475,"Mikasa volleyball",5,1200,"/sc/volley.jpg");
67  DNode NEW13= new DNode();
68  NEW13.setAll(50778,"Wilson basketball",5,1500,"/sc/wilson.jpg");
69  LAST=NEW13;
70
71  NEW1.BACK=null;NEW1.FORW=NEW2;
72  NEW2.BACK=NEW1;NEW2.FORW=NEW3;
73  NEW3.BACK=NEW2;NEW3.FORW=NEW4;
74  NEW4.BACK=NEW3;NEW4.FORW=NEW5;
75  NEW5.BACK=NEW4;NEW5.FORW=NEW6;
76  NEW6.BACK=NEW5;NEW6.FORW=NEW7;
77  NEW7.BACK=NEW6;NEW7.FORW=NEW8;
78  NEW8.BACK=NEW7;NEW8.FORW=NEW9;
79  NEW9.BACK=NEW8;NEW9.FORW=NEW10;
80  NEW10.BACK=NEW9;NEW10.FORW=NEW11;
81  NEW11.BACK=NEW10;NEW11.FORW=NEW12;
82  NEW12.BACK=NEW11;NEW12.FORW=NEW13;
83  NEW13.BACK=NEW12;NEW13.FORW=null;
84
85  }
86
87  }
88
89

```

การทำงานของอัลกอริทึม

ดังที่เคยกล่าวไว้ในโปรเจกต์แรกว่า จะทำหน้า Login ออกมาให้สมบูรณ์แบบ จะเห็นว่า ในหน้า DataStore จะมีการสร้างคลาสถึง 2 คลาส นั่นก็คือ node และ DNode ค่ะ เนื่องจากว่า หน้า Login หากมีการสลับหน้าไปมา ข้อมูลที่ผู้ใช้ sign up จะหายไปหมด ซึ่งจะต้องทำการ ลงทะเบียนใหม่ทุกครั้งที่มีการสลับหน้ามายัง Login เพจ เพื่อตัดปัญหาตรงนี้ หนูจึงจัดเก็บข้อมูล ที่ผู้ใช้ลงทะเบียนแล้ว ไว้ที่ไฟล์กลางก็คือ DataStore จึงต้องสร้างคลาส node ที่จับเก็บไอดี पास เวิร์ดของผู้ใช้ไว้ที่นี้ และคลาส DNode ที่จะสร้างอัลกอริทึมของอาจารย์ในลำดับถัดไปค่ะ



1. กำหนดให้คลาส node มีตัวเชื่อมแบบ 1 ทาง ก็คือ node link ทุกครั้งที่ผู้ใช้ทำการ sign up เข้ามา ระบบจะดึงข้อมูล String จาก jTextField1 มาจัดเก็บไว้ที่ NEW.id และ String จาก jTextField8 เข้ามาจัดเก็บไว้ที่ NEW.pass หลังจากนั้น ข้อมูลทั้งอ็อบเจกต์จะเข้ามาจัดเก็บไว้ที่จุดเริ่มต้นของลิงค์ลิสต์ทุกครั้ง หนูประกาศอ็อบเจกต์ Head ไว้ที่ DataStore เวลาเข้าถึงที่ Head จะต้องใส่ DataStore.Head ทุกครั้ง จะเป็นการจัดเก็บไว้ที่ไฟล์กลาง ซึ่ง Head จะทำหน้าที่เดียวกันกับ START คือเป็นจุดเริ่มต้นในการท่อง โหนด ผู้ใช้มีการสร้างไอดีไว้ที่ชุด เวลา login

ระบบก็จะท่องโนดตามจำนวนที่มีอยู่ จนกว่าจะเจอ หากเจอชุดข้อมูลที่ตรงกัน ก็อนุญาตให้เข้าหน้าถัดไปได้

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    String user=JTextField1.getText();
    String pass=JPasswordField1.getText();

    node PTR=DataStore.Head;
    while(PTR!=null){
        if(user.equals((String)PTR.id) && pass.equals(PTR.pass)){
            System.out.println("Login complete!");
            JOptionPane.showMessageDialog(this, "สวัสดีครับสำเร็จ!");
            Menu move = new Menu(this); // สร้างหน้าฟอร์มใหม่
            move.setVisible(true); // แสดง Form2
            this.setVisible(false);
            return;
        }
        PTR=PTR.link;
    }
    System.out.println("not found");
    JOptionPane.showMessageDialog(this, "user or password not correct");
}
```

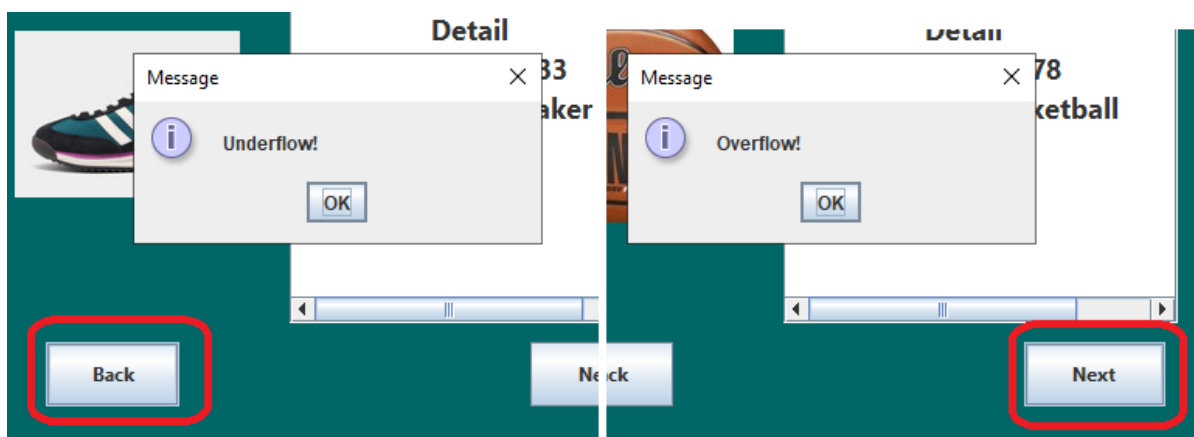
Handwritten notes:
 - `int i = 0;` (next to `PTR=DataStore.Head`)
 - `i < n` (next to `while(PTR!=null)`)
 - `if(user.equals((String)PTR.id) && pass.equals(PTR.pass))` (circled in blue)
 - `Menu move = new Menu(this);` (circled in green)
 - `move.setVisible(true);` (circled in green)
 - `this.setVisible(false);` (circled in green)
 - `PTR=PTR.link;` (circled in red)
 - `return;` (circled in green)
 - `System.out.println("not found");` (circled in green)
 - `JOptionPane.showMessageDialog(this, "user or password not correct");` (circled in green)

2. หน้า Menu ปกติจะมีตัวแปร current จัดเก็บ integer ตำแหน่งปัจจุบันที่โชว์ค่าอยู่ แต่สำหรับการทำแบบลิงคิลิส จะไม่สามารถเข้าถึงตำแหน่ง index ได้ จึงต้องกำหนดให้ current ในโปรเจกต์นี้มี data type คือ คลาส DNode เพื่อสำหรับจัดเก็บอ็อบเจกต์ที่แสดงผลปัจจุบันอยู่แทน เวลาค้นหาหรือท่องโนด ก็สามารถส่งอ็อบเจกต์ท่องโนด ซึ่งก็คือ PTR ให้เท่ากับ LOC แล้ววีธีรัน LOC ซึ่งเป็นอ็อบเจกต์ตำแหน่งที่ต้องการ กลับไป assign ให้กับ current ได้เลย เพื่อทำการโชว์ข้อมูลต่อไป

```
public DNode SEARCHTYPE(int item) {
    int ITEM = item;
    DNode LOC=null;
    DNode PTR=null;
    PTR=START;
    while(PTR!=null){
        if(ITEM==(int)PTR.type){
            LOC = PTR;
            return LOC;
        }
        PTR=PTR.FORW;
    }
    return LOC;
}
```

Handwritten notes:
 - `int ITEM = item;` (next to `int ITEM = item;`)
 - `DNode LOC=null;` (next to `DNode LOC=null;`)
 - `DNode PTR=null;` (next to `DNode PTR=null;`)
 - `PTR=START;` (next to `PTR=START;`)
 - `while(PTR!=null){` (next to `while(PTR!=null){`)
 - `if(ITEM==(int)PTR.type){` (next to `if(ITEM==(int)PTR.type){`)
 - `LOC = PTR;` (next to `LOC = PTR;`)
 - `return LOC;` (next to `return LOC;`)
 - `else` (next to `else`)
 - `PTR=PTR.FORW;` (next to `PTR=PTR.FORW;`)
 - `return LOC;` (next to `return LOC;`)

3. ปุ่ม Back และ Next เดิมที่ต้องกำหนดขอบเขตว่าต้องไม่น้อยกว่า LB และไม่เท่ากับ UB ถึงสามารถ +- current เพื่อโชว์ข้อมูลต่อได้ แต่สำหรับลิงค์ลิส จะเปลี่ยนเป็นการตรวจสอบว่าตำแหน่งด้านซ้ายของ current ซึ่งก็คือ current.BACK ขึ้นไปที่ null หรือไม่ หากใช่แสดงว่าตำแหน่งปัจจุบันที่ current แสดงผลอยู่คือรายการแรกของลิงค์ลิสแล้ว ไม่สามารถที่จะขยับไปยังตัวก่อนหน้าได้อีก ในทำนองเดียวกัน หาก current.FORW ที่เก็บตำแหน่งลิสตัวถัดไป ขึ้นไปที่ null ด้วย นั่นหมายความว่า ตำแหน่ง current ที่แสดงผลอยู่เป็นลำดับสุดท้ายแล้ว ไม่สามารถขยับไปยังตัวถัดไปได้นั่นเอง จะแสดง Pop up ให้ผู้ใช้ทราบว่า Underflow และ Overflow



4. สำหรับการ search บนหน้า Manage จะเพิ่มขั้นตอนการตรวจสอบข้อความด้วยว่า text ที่ผู้ใช้ input เข้ามานั้น เป็นตัวเลขจริงหรือไม่ โดยการสร้างฟังก์ชัน checkValue(Object value) ไว้ การทำงานของฟังก์ชันจะตรวจสอบว่า string ที่รับมาจาก TextField นั้น สามารถแปลงมาเป็น Integer ได้หรือไม่ หากแปลงได้นั้นหมายความว่า เเทคดังกล่าวเป็นตัวเลข แต่ถ้าแปลงไม่สำเร็จ ก็

จะทราบได้ว่าผู้ใช้กรอกตัวอักษรเข้ามา ก็จะทำให้การคืนค่า res ซึ่งกำหนดไว้ตั้งแต่แรกมาให้เท่ากับ -1 กลับไป แล้วทำการเข้า if เพื่อตรวจสอบว่าแปลงเป็น integer สำเร็จหรือไม่ หากแปลงสำเร็จ เราก็จะได้ค่า code ของสินค้าที่ต้องการค้นหาเรียบร้อยแล้ว

```

public int checkValue(Object value) {
    int res=-1;
    if (value instanceof String) {
        String str = (String) value;

        try {
            // ถ้าแปลงเป็นเลขได้ แสดงว่าไม่ใช่ pure string
            res=Integer.parseInt(str);
            System.out.println("This is a number: " + str);
        } catch (NumberFormatException e) {
            // ถ้าแปลงไม่ได้ = เป็นข้อความ
            System.out.println("This is a String: " + str);
            return res; // ออกจากฟังก์ชัน
        }
    } else {
        // กรณีเป็น object อื่นๆ
        System.out.println("Not a string: " + value);
    }
    return res;
}

```

Handwritten notes in Thai:

- ตรวจสอบว่าแปลง int ได้ไหม (Check if it can be converted to int)
- แปลงไม่ได้ คืนค่าเริ่มต้น (If conversion fails, return default value)

หลังจากได้เลขที่ต้องการค้นหาแล้ว เราก็จะทำการโยนเลข code ดังกล่าวลงบนอาณูเมนบนฟังก์ชัน SEARCH() ซึ่งฟังก์ชันนี้จะทำการสร้างอ็อบเจกต์ขึ้นมา 2 ตัวดังที่เคยกล่าวไว้ก่อนหน้านี้ก็คือ PTR (ตัวท่องโนด) และ LOC (ตัวเก็บผลลัพธ์) เราจะสั่งให้ PTR เข้าไปในลูป While ในการหาตำแหน่งของอ็อบเจกต์ที่จัดเก็บข้อมูล code ที่ใส่มาในพารามิเตอร์ ว่า PTR.code ในแต่ละรอบนั้นตรงกันกับ code ที่ใส่เข้ามาค้นหาหรือไม่ หาก PTR ท่องโนดจนเจอว่าตำแหน่งถัดไปเป็น null แสดงว่า PTR ถึงตัวสุดท้ายเรียบร้อยแล้ว จะไม่มีตัวถัดไป นั่นหมายความว่าเราไม่เจอเลข code ดังกล่าวนั่นเอง ระบบก็จะดึงออกจากฟังก์ชัน แต่ถ้า PTR.code ตรงกับ code ที่ใส่เข้ามาในรอบใดๆของลูป While ก็แสดงว่ามีเลขดังกล่าวอยู่ภายในลิส ก็จะกำหนดให้ LOC เก็บผลลัพธ์การค้นหาให้เท่ากับอ็อบเจกต์ PTR ในตำแหน่งนั้นๆ แล้วทำการคืนค่า LOC กลับไป

```

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    SearchCode = checkValue(jTextField1.getText());
    if (SearchCode != null) {
        System.out.println("Input number code need to search");
        JOptionPane.showMessageDialog(this, "กรุณากรอกตัวเลข code ที่ต้องการค้นหา");
    }
    if (SearchCode == null) {
        JOptionPane.showMessageDialog(this, "ไม่มีเลข code สอดคล้อง");
    }
    DefaultTableModel model = (DefaultTableModel) jTable1.getModel();
    model.setRowCount(0);

    model.addRow(new Object[] { "-",
        temp.code,
        temp.name,
        temp.price,
        temp.type,
        temp.pic
    });
    System.out.println("Status : Search complete");
}

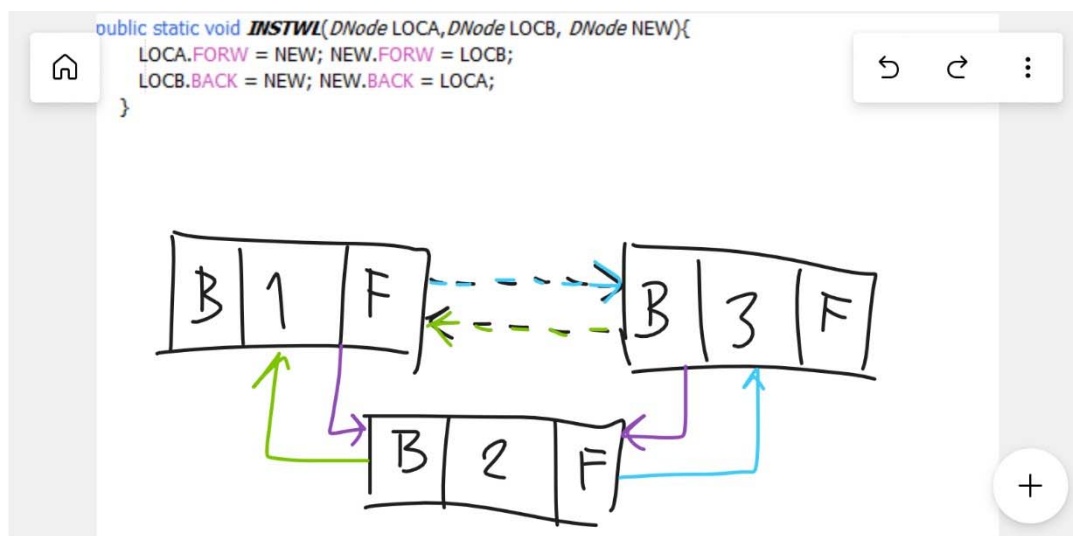
public int SEARCH(Object item) {
    int ITEM = (int)item;
    int LOC=null;
    int PTR=START;
    while (PTR!=null) {
        if (ITEM==((int)PTR.code)) {
            LOC=PTR;
            return LOC;
        }
        PTR=PTR.getNext();
    }
    return LOC;
}

```

Handwritten notes in Thai:

- ไม่เจอ (Not found)
- ไม่เจอ (Not found)
- show data ที่เจอ (Show data that was found)
- อย่าลืม set i=0 (Don't forget to set i=0)
- ต้องจบที่ตัวสุดท้าย (Must end at the last item)
- ก็พ่วงกัน ก็เก็บค่าที่พ่วงกัน (It's linked, so store the linked value)
- ไม่ลอปายไปต่อ (i++) (Don't loop again (i++))

5. การ Insert เดิมที่เราจะให้ผู้ใช้กรอกเลข index เข้ามา เพื่อแทรกข้อมูลเข้าไประหว่างกลาง แต่สำหรับโปรเจกต์ลิงก์ลิสต์ เราไม่สามารถเข้าถึงตำแหน่งตรงๆ ได้ จึงจำเป็นต้องเปลี่ยนไปให้ผู้ใช้กรอกเลข code ของสินค้าที่มีอยู่ เราถึงสามารถใช้คุณสมบัติของ .FORW เพื่อแทรกข้อมูลเข้าไปต่อท้ายของสินค้านั้นๆ ได้อย่างง่ายดายเลย แต่อย่าลืมว่า กรณีที่ผู้ใช้ต้องการแทรกข้อมูลเข้าไปตัวแรกล่ะ จะทำยังไง... ถ้าจะบอกว่าเปลี่ยนใหม่ ให้แทรกข้อมูลเข้าไปต่อหน้าของข้อมูล code ที่ผู้ใช้พิมพ์เข้ามาแทน คำถามต่อมาคือ แล้วถ้าผู้ใช้ต้องการแทรกเข้าไปเป็นตัวสุดท้ายล่ะ จะทำยังไง... เพื่อแก้ไขปัญหาดังกล่าวนี้ หนูจึงสร้าง TextField ขึ้นมา 2 ช่อง เพื่อให้ผู้ใช้เลือกว่า จะต่อหน้าหรือต่อท้าย ซึ่งเราจะใช้อัลกอริทึมเดียวกันของอาจารย์ แต่เพียงแค่เปลี่ยนพารามิเตอร์เขยิบไปตามก่อนหรือหลังเท่านั้นเอง เช่น ต่อท้ายจะใช้ (A,A.FORW,B) ต่อหน้าก็จะใช้ (A.BACK,A,B) แทน เป็นต้น พอเข้าใจกันแล้ว หนูจะมีพารามิเตอร์ตัวก่อนหน้าและตัวถัดไปของ NEW สามารถกำหนดค่าไปให้ LOCA.FORW = NEW ให้ตัวก่อนหน้าชี้มาที่ตัวใหม่ได้เลยและ NEW.FORW = LOCB ตัวใหม่ชี้ไปตัวถัดไป รวมทั้งขากลับ กำหนดให้ ตัวถัดไปชี้กลับมาที่ตัวใหม่ LOCB.BACK = NEW ส่วนตัวใหม่ชี้กลับไปยังตัวก่อนหน้า NEW.BACK = LOCA



6. การ Delete พาร์ทนี้ทำให้หนูตาสว่าง ว่าทำไมลิงก์ลิสต์ถึงต้องมี 2 ทาง ปกติถ้าทำแบบลิงก์ลิสต์ธรรมดาทางเดียว เวลาลบโนด เราต้องสร้างอ็อบเจกต์สำหรับเก็บตัวก่อนหน้า PTR ด้วย เช่น PREV, LOCP เวลาทวงโนด เราก็จะขยับ PTR และตัวก่อนหน้า ไปพร้อมๆ กัน พอถึงตำแหน่ง

เป้าหมาย ถึงจะกำหนดให้ PREV หรือ LOCP ซ้ำเข้าไปยังตัวที่ PTR ซ้ำตัวถัดไป แต่สำหรับลิงค์
ลิส 2 ทาง เราจะมี .BACK หรือ Attribute ที่จัดเก็บตัวก่อนหน้ามาจัดการปัญหาตรงนี้แทน เราไม่
ต้องสร้าง PREV หรือ LOCP คอยไล่โนดไปพร้อม PTR อีกต่อไป

เริ่มแรก การลบโนดทำวิธีเดียวกันกับ Insert เลย คือค้นหาตำแหน่งที่ต้องการก่อน แล้ว
จึงโยนเข้าฟังก์ชัน แต่สำหรับการลบนั้น ไม่จำเป็นต้องสร้าง TextField 2 อันสำหรับให้ผู้ใช้เลือก
ว่าจะลงหน้าหรือต่อท้ายอีกแล้ว แต่จะเป็นการสร้างเงื่อนไขเพิ่มเติมแทน กรณีที่ code ที่ผู้ใช้
กรอกเข้ามา ค้นหาแล้วส่งค่า LOC กลับมาตรงกับตำแหน่งตัวแรกของโนดลิส เราจะสั่งให้
LOC.FORW.BACK = null; ซึ่งก็คือ ให้ Attribute BACK ของตัวถัดไปจาก LOC นั้น ให้ชี้ไปที่
ว่าง null เพราะเราจะทำการลบตัว LOC ดังนั้น ตัวถัดไปจาก LOC ซึ่งก็คือ LOC.FORW จะ
กลายเป็นตัวแรกแทน แล้วจึงกำหนดให้ START ชี้ไปยังตัวถัดไปแทน ในทำนองเดียวกัน หาก
ผู้ใช้กรอก code ที่ต้องการลบตรงกันกับตำแหน่งสุดท้าย เราก็ต้องสั่งให้ LOC.BACK.FORW =
null; ซึ่งก็คือ Attribute ที่เก็บตัวถัดไปของ โนดก่อนหน้าที่เราจะลบชี้ไปที่ว่าง null เพราะเราจะ
ลบตัวสุดท้าย จำเป็นต้องกำหนดให้ตัวก่อนหน้าเป็นตัวสุดท้ายแทน แล้วจึงกำหนดค่าให้ LAST
เท่ากับตัวก่อนหน้าแทน

```
java.lang.String type = LOC.type;
jLabel15.setText("Type : "+LOC.type);
if(LOC==START){
    *LOC.FORW.BACK* = null;
    START=LOC.FORW;
    DataStore.r--;
    System.out.println("Status : delete complete!");
    jLabel11.setText("Status : delete complete!");
    jLabel21.setText("Amount : "+DataStore.r);
    loadTableData();
    return;
}
if(LOC==LAST){
    *LOC.BACK.FORW* = null;
    START=LOC.BACK;
    DataStore.r--;
    System.out.println("Status : delete complete!");
    jLabel11.setText("Status : delete complete!");
    jLabel21.setText("Amount : "+DataStore.r);
    loadTableData();
    return;
}
DELTWL(LOC);
```

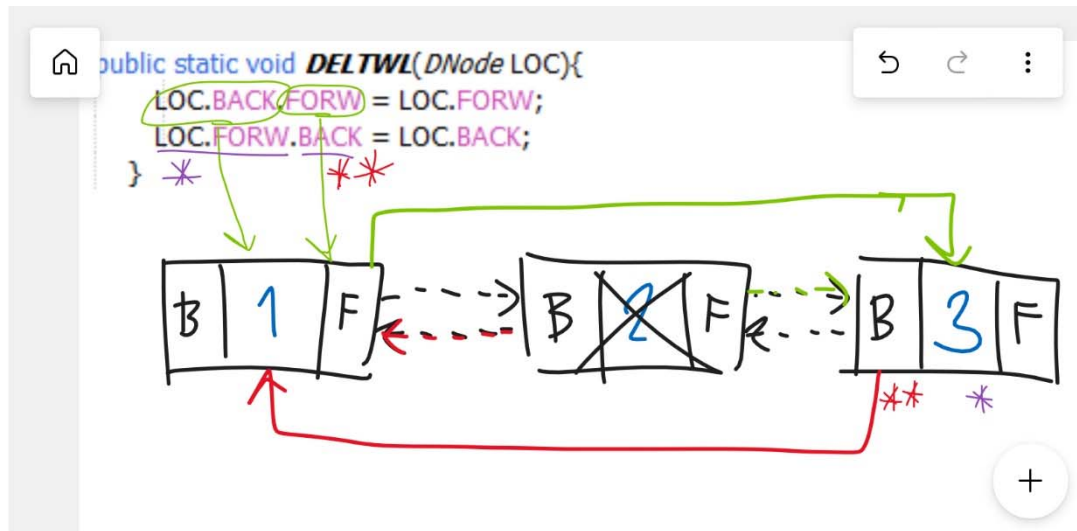
1. กรณีลบตัวแรกสุด

2. กรณีลบตัวสุดท้าย

3. กรณีลบช่วงกลางโนด

กรณีสุดท้าย หากผู้ใช้กรอก code เข้ามาตำแหน่งช่วงระหว่างกลาง เราสามารถโยนค่า
LOC เข้าไปในฟังก์ชันอาจารย์ซึ่งก็คือ DELTWL(LOC); ได้เลย การลบช่วงกลางของโนด

จะต้องระวังเป็นอย่างมาก หากลำดับการลบผิด อาจจะทำให้เราหาเส้นที่เหลือไม่เจอ ดังนั้น
 ขั้นตอนที่ 1. กำหนดให้ LOC.BACK.FORW = LOC.FORW; เพื่อสั่งให้ตัวก่อนหน้าชี้ข้ามไปยัง
 ตัวถัดไปของตัวที่จะลบ 2. LOC.FORW.BACK = LOC.BACK; สั่งให้ .BACK ของตัวถัดไปชี้
 ข้ามมายังตัวก่อนหน้าตัวที่จะลบ



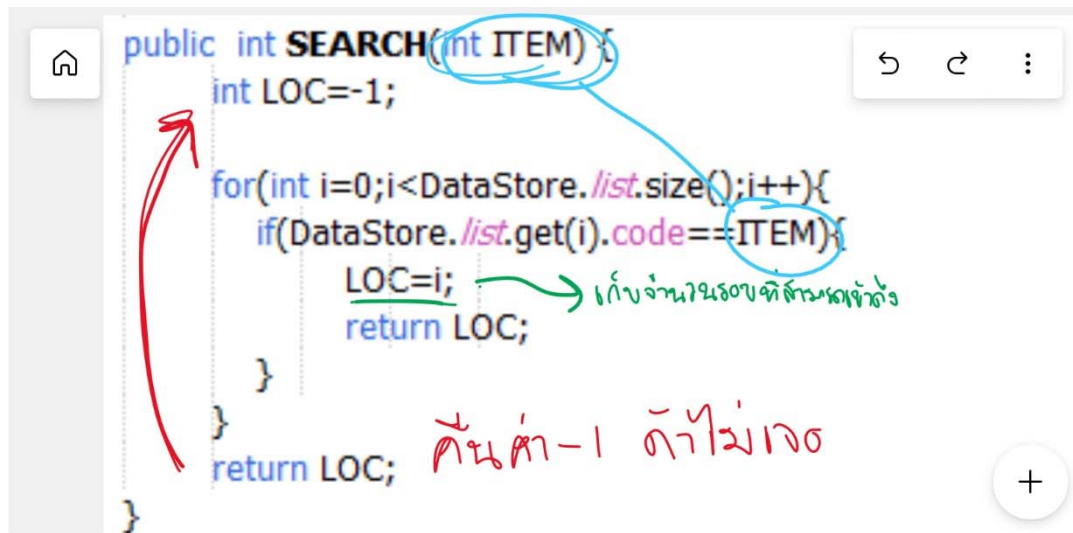
JAVA.UTIL.LINKEDLIST

การใช้งานแพ็คเกจ java.util.linkedlist จะมีลักษณะคล้ายๆกับ ArrayList ซึ่งโปรเจกต์นี้เราไม่จำเป็นต้องใช้ตัวเชื่อม BACK หรือ FORW อีกต่อไป

เริ่มต้น การสร้างข้อมูลเราไม่ต้องมาไล่เทียบทีละตัวแล้ว สามารถ add เข้าไปได้เลย

```
static {  
  
    list.add(new DNode(10233, "Adidas sneaker", 1, 4600, "/sc/sneakerAdi.jpg"));  
    list.add(new DNode(10472, "Nike sneaker", 1, 4200, "/sc/sneakerNike.jpg"));  
    list.add(new DNode(10785, "Skecher sneaker", 1, 2500, "/sc/ske.jpg"));  
    list.add(new DNode(20096, "Umbro Glove", 2, 800, "/sc/ub.jpg"));  
    list.add(new DNode(20504, "Nike football shirt", 2, 1500, "/sc/shirtNike.jpg"));  
    list.add(new DNode(30319, "Ping stick golf", 3, 1300, "/sc/golfstick.jpg"));  
    list.add(new DNode(30873, "Anyday golf bag", 3, 5000, "/sc/golfbag.jpg"));  
    list.add(new DNode(40152, "Yonex strike", 4, 2400, "/sc/yon.png"));  
    list.add(new DNode(40197, "Kawasaki strike", 4, 1700, "/sc/kawa.jpg"));  
    list.add(new DNode(40601, "Victor shuttlecock", 4, 200, "/sc/shuttle.jpg"));  
    list.add(new DNode(50290, "Puma football", 5, 800, "/sc/puma.jpg"));  
    list.add(new DNode(50475, "Mikasa volleyball", 5, 1200, "/sc/volley.jpg"));  
    list.add(new DNode(50778, "Wilson basketball", 5, 1500, "/sc/wilson.jpg"));  
  
}
```

1. การ search จากเดิมทีในโปรเจกต์ลิงค์ลิส เราจะต้องสร้าง LOC สำหรับจัดเก็บอ็อบเจกต์ผลลัพธ์ แต่สำหรับแพ็คเกจนี้ เราจะให้ LOC เก็บ integer แทน เพราะเราสามารถเข้าถึง index ได้แบบอ้อมๆ ซึ่งจริงๆแล้วไม่ได้เป็นการเข้าถึง index ตรงๆซะทีเดียว แต่เป็นการนับจำนวนรอบที่วนเจอข้อมูลค่ะ โดยการสั่งให้โปรแกรมทำการดูป ตามจำนวนที่อ็อบเจกต์มีอยู่ $i < \text{DataStore.list.size()}$ แล้วไปไล่เทียบกับข้อมูล สำหรับโปรเจกต์นี้จะไล่เทียบกับ code เหมือนเดิมที่ผู้ใช้อยู่กรอกเข้ามา หากเจอที่ตำแหน่งไหน ก็กำหนดให้ LOC เท่ากับ i หรือตำแหน่งนั้นๆ



2. การ Insert เนื่องจากเราสามารถเข้าถึงตำแหน่งได้ คล้ายๆกับ index ซึ่งปกติแล้ว LinkedList จะไม่มีตำแหน่ง index แบบปกติให้เราเข้าถึงได้โดยตรง ซึ่งแพ็คเกจนี้จะมีคำสั่ง `listIterator()` เพื่อตรวจสอบตำแหน่งเช่นเดียวกันกับ index ซึ่งปกติถ้าเรามี Attribute `FORW` เราสามารถกำหนดแทรกที่นี่ได้เลย แต่โปรเจกต์นี้เราดึงแพ็คเกจเข้ามาใช้งาน จึงควรใช้คำสั่งที่เขามีให้ในการแทรกข้อมูลจะดีกว่า ในทางกลับกัน หากเราจะใช้คำสั่งจากแพ็คเกจในการแทรกข้อมูล `.add` ต่อจาก `listIterator()` เราจำเป็นต้องสร้างลูปอีก จะถือเป็นการเพิ่ม Big O โดยไม่จำเป็น ในเมื่อโปรเจกต์นี้มีอิสระในการใช้โค้ด ดังนั้น เราสามารถสร้าง `TextField` ที่จัดเก็บตำแหน่งที่ผู้ใช้ต้องการแทรกได้ช่องเดียวก็พอ หลังจากนั้นก็นำเลขที่ผู้ใช้กรอก ไปตรวจสอบว่าเป็นตัวเลขหรือไม่ แล้วทำการ `SEARCH` ตามลำดับ เราจะได้เลขตำแหน่งที่กลับมา `IST` แล้วเรียกฟังก์ชัน `add(IST-1, E e)` เพื่อแทรกข้อมูลลงในตำแหน่งที่ `IST-1` ได้เลยล่ะ

```
list.add(IST-1, new DNode(Code, jTextField2.getText(), Type, Price, "/sc/home.png"));
```

3. การ delete ใช้หลักการเดียวกันเลยล่ะ คือค้นหาตำแหน่งก่อน แล้วจึงใช้คำสั่ง `.remove()`

```

int delCode = checkValue(jTextField5.getText());
if(delCode==-1){
    System.out.println("Input number need to delete at Delete code");
    JOptionPane.showMessageDialog(this, "กรุณารอกตัวเลขที่ต้องการลบในช่อง cc");
    return;
}
int LOC=SEARCH(delCode);
if(LOC==-1){
    System.out.println("not found");
    JOptionPane.showMessageDialog(this, "ไม่มีเลข code ดังกล่าวค่ะ");
    return;
}
DNode dd=list.remove(LOC);
jLabel12.setText("Code : "+dd.code);
jLabel13.setText("Name : "+dd.name);
jLabel14.setText("Price : "+dd.price);
jLabel15.setText("Type : "+dd.type);

System.out.println("Status : delete complete!");
jLabel11.setText("Status : delete complete!");
jLabel21.setText("Amount : "+DataStore.list.size());
loadTableData();

```

จะเห็นว่าทั้ง `java.util.LinkedList` กับ `ArrayList` คล้ายคลึงกันมาก ซึ่งจริงๆแล้ว โปรเจคนี้ควรเพิ่มข้อมูลจากต้นลิสหรือท้ายลิส ถึงจะยังคงคอนเซ็ปเอกลักษณะของลิงก์ลิสไว้มากกว่า แต่เนื่องจากโปรเจคนี้ เป็นการดัดแปลงมาจากโปรเจคอาเรย์เดิม ซึ่งได้สร้างฟังก์ชันการเพิ่มข้อมูลแบบแทรกข้อมูลเอาไว้ จึงจำเป็นต้องใช้อัลกอริทึมเดียวกันมาใช้งาน อย่างไรก็ตาม ไม่ว่าจะใช้อัลกอริทึมใด สุดท้ายแล้วนักศึกษาก็สามารถทราบถึงความแตกต่างและวิธีใช้งานของโครงสร้างแต่ละแบบ รวมถึงแพ็คเกจ `util` ของทุกๆ โครงสร้างด้วยว่าควรนำไปปรับใช้อย่างไร

ขอบคุณค่ะ

A decorative flourish consisting of symmetrical, swirling lines that frame the text. In the center, there is a small, stylized floral or leaf-like motif.